Reproducibility Report for Expectation-driven sensory adaptations support enhanced acuity during categorical perception by Tim Sainburg et. al. (2025, Nature Neuroscience)

Will Hammond (whammond@ucsd.edu)

2026-11-08

Table of contents

Introduction . . . 1
Links . . . 2
Methods . . . 2
Description of the steps required to reproduce the results . . . 2
Differences from original study . . . 3
Project Progress Check 1 . . . 3
Measure of success . . . 3
Pipeline progress . . . 3
Results . . . 4
Data preparation . . . 4
Exploratory analyses . . . 15
Discussion . . . 15
Summary of Reproduction Attempt . . . 15
Commentary . . . 15

Introduction

Tim Sainburg's 2025 paper includes a breadth of modern techniques and analysis used in the Gentner Lab, making it a useful paper to simulate in order to familiarize myself with relevant concepts and data. I will be simulating behavioral data from a two-alternative choice task (2AC) where the choices are modulated by the reliabilty of an auditory primer cue. My main analysis goal is to find a significant difference in the population psychometric curve inflection

points for decision making based on the reliability of the primer cue. This simulation will engage my interest in sensory systems while allowing me to work with real data and get experience analyzing it.

For this project, I will need to simulate a new data set, clean it to calculate the necessary analytic points (such as change in inflection point), create a linear mixed effects model, and create a 2 distribution histogram (figure 3e in the original paper). I will be aiming to replicate analysis with statistics similar to those reported in the paper ($= -11.12$, s.e.m. $= 1.158$, $z = -9.6$, $P < 0.001$) I believe that this will be an adequate challenge because it is a large dataset, and I do not have any experience with linear mixed effects models. I will need to ensure that I am cleaning and analyzing such a large dataset accurately and take the necessary steps to understand linear mixed effects models. Another challenge will be choosing simulation parameters that accuratley model the behavioral data. Although the data is not available in this paper, it is available in another paper (Gorman et al 2026) that analyzes the same dataset, making it easier to simulate an accurate representation of the data.

Learning Goals: My main goal is to re-familiarize myself with data analysis in R. I have used the language a few times for smaller projects, but without consistent use, I stil do not feel confident in my programming abilities. Although I anticipate challenges due to my inexperience, my conceptual understanding of the data, paper, and general programming knowledge make the scope of this project feasible for me. I also want to use this project as an opportunity to gain a greater understanding of linear mixed effects models, what they can tell us, and how to use them.

Links

Project repository: https://github.com/willhammond03/sainburg2026.git

Original paper: https://doi.org/10.1038/s41593-025-01899-1

Github repository for paper with accessible data: https://github.com/juliagorman/anticipatory_geometry_paper.git

Methods

Description of the steps required to reproduce the results

To reproduce the key result from this study, I will first need to change the data into a usable form. The pickle files need to be converted into files that R can use and stored in the repository instead of a local machine. I will need to create a data frame from these individual data files, and this data frame will benefit from being cleaned to remove extraneous variables. I will need to keep the “cue prob,” “correct,” “interpolation point,” and other values that I will need to discuss with the author to reduce ambiguity. After cleaning, I will need to simulate new data

based on the parameters of the real data. I will keep experimental values the same, and only change response values based on parameters observed in the original data. After simulating, I will make a new dataframe for the the dual histogram from figure 3e. This figure is strictly for interpretation purposes and does not have associated statistical tests. Axes in the paper are imprecise, so confirmation of replication will need to be done visually. Interpolation is defined as the degree of the model generated mixture of auditory stimuli based on the 128 syllable continuum. Next, I will make a data frame to recreate figure 3f, including probability of accurate cue and inflection value. This visualization includes individual trial data, averaged individual data, averaged population data, and standard population level error. I will need to design a pipeline to include these statistics. I will need to fit a trendline using a linear mixed effects model. The model tests psychometric inflection point changing as a result of the main effect of cue probability while accounting for random effects of subject and cue response per subject. I plan on calculating the same statistics from the linear mixed effects model that are reported in the paper: cue probability: $= -11.12$, s.e.m. $= 1.158$, $z = -9.6$, $P < 0.001$.

Differences from original study

The original study runs these analyses and visualizations in Python, meaning that I will need to rebuild the code for it to fit into my analysis in R. I anticipate that this difference may change the appearance and aesthetics of my data visualizations, but I do not anticipate it significantly changing the statistical values returned. Running a simulation introduces structured variation, so I anticipate values in my figures and statistics to look slightly different, but not to an extent that alters the findings to a significant degree. The large effect size and the small p-value lead me to believe that simulating the data will not significantly change the statistical returns.

Project Progress Check 1

Measure of success

I will measure success by comparing the simulated to the original confidence interval. Additionally, I will analyze the extent to which a shift could occur if the bird was not influenced by the prior cue.

Pipeline progress

Earlier in this report, you described the steps necessary to reproduce the key result(s) of this study. Please describe your progress on each of these steps (e.g., data preprocessing, model fitting, model evaluation).

Right now, I have roughly recreated the 3 figures from the paper I am replicating. I am still working out some differences between the figures in the paper and my replications. I will also

need to run a linear mixed effects model to replicate the reported effect size in the paper. After these steps, I will need to create a simulated dataset without the reported main effect to see the differences in the figures and statistics. There is still a lot of work to be done, but having a functioning data pipeline that recreates the targeted figures is a big step forward.

Results

Data preparation

Data preparation following the analysis plan.

```
#cleaning for 20 graph array

#### Load Relevant Libraries and Functions
library(here)
```

```
here() starts at /Users/whammond/Documents/GitHub/sainburg2026
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2

-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
#### Function that cleans one bird's raw file
clean_bird <- function(path) {
  bird_id <- tools::file_path_sans_ext(basename(path))

  d <- read_csv(path, show_col_types = FALSE) |>
    filter(cueing) |>   # experimental phase only
    mutate(
      p_signal_right = case_when(
```

```
        cue_class == "NC"     ~ 0.5,          # no-cue condition
        cue_direction == "R"  ~ cue_prob,
        cue_direction == "L"  ~ 1 - cue_prob,
        TRUE                  ~ NA_real_      # CN (uninformative cue) dropped
      ),
      # P(reinforce left | cue)
      p_signal_left = round(1 - p_signal_right, 3),
      bird = bird_id
    ) |>
    filter(!is.na(p_signal_left), !is.na(response_bool))

  # Put every morph in the same orientation (flip any morph that falls)
  x_min  <- min(d$interpolation_point)
  x_max  <- max(d$interpolation_point)
  x_span <- x_max - x_min

  morph_direction <- d |>
    group_by(interpolation) |>
    summarise(
      low_end  = mean(response_bool[interpolation_point <= x_min + x_span * 0.25]),
      high_end = mean(response_bool[interpolation_point >= x_max - x_span * 0.25]),
      .groups = "drop"
    ) |>
    mutate(flip = low_end > high_end)

  d |>
    left_join(select(morph_direction, interpolation, flip), by = "interpolation") |>
    mutate(
      interpolation_point = if_else(flip, x_min + x_max - interpolation_point,
                                    interpolation_point),
      n_flipped_morphs = sum(morph_direction$flip)
    ) |>
    # keep only what the plot needs, so column types match across birds
    select(bird, interpolation, interpolation_point, cue_class, cue_prob,
           cue_direction, p_signal_left, response_bool, n_flipped_morphs)
}

#### clean all raw files
raw_files <- list.files(here("data"), pattern = "^B\\d+\\.csv$", full.names = TRUE)

df_all <- map(raw_files, clean_bird, .progress = TRUE) |>
  list_rbind()
```

```
 ========>-----------------------   25% |  ETA:  6s
 ====================>----------   65% |  ETA:  3s
```

```
dir.create(here("data", "clean"), showWarnings = FALSE)
write_csv(df_all, here("data", "clean", "all_birds_clean.csv"))
```

```
# plotting 20 graph array

#### proportion of right pecks per bird, cue condition, and point
plot_data <- df_all |>
  group_by(bird, p_signal_left, interpolation_point) |>
  summarise(
    p_right  = mean(response_bool),
    n_trials = n(),
    .groups  = "drop"
  )

#### panel order
bird_order <- c("B1174","B1426","B1170","B1432","B1437",
                "B1433","B1597","B1427","B1188","B1244",
                "B1590","B1399","B1591","B1252","B1110",
                "B1390","B1248","B1593","B1595","B1276")

plot_data <- plot_data |> mutate(bird = factor(bird, levels = bird_order))

#### logistic fit, one fit per bird x cue
fit_logistic <- function(d) {
  nls(
    p_right ~ p_max + (p_min - p_max) / (1 + (interpolation_point / inflection)^slope),
    data = d,
    weights = n_trials,
    start = list(p_min = 0.1, p_max = 0.9, inflection = 64, slope = 10),
    algorithm = "port",
    lower = c(p_min = 0,   p_max = 0.5, inflection = 1,   slope = 0.1),
    upper = c(p_min = 0.5, p_max = 1,   inflection = 128, slope = 100)
  )
}

fits <- plot_data |>
  group_by(bird, p_signal_left) |>
  nest() |>
  mutate(model = map(data, possibly(fit_logistic, otherwise = NULL)))
```

```r
# Which fits failed to converge?
fits |>
  filter(map_lgl(model, is.null)) |>
  select(bird, p_signal_left) |>
  print(n = Inf)
```

```
# A tibble: 0 x 2
# Groups:   bird, p_signal_left [0]
# i 2 variables: bird <fct>, p_signal_left <dbl>
```

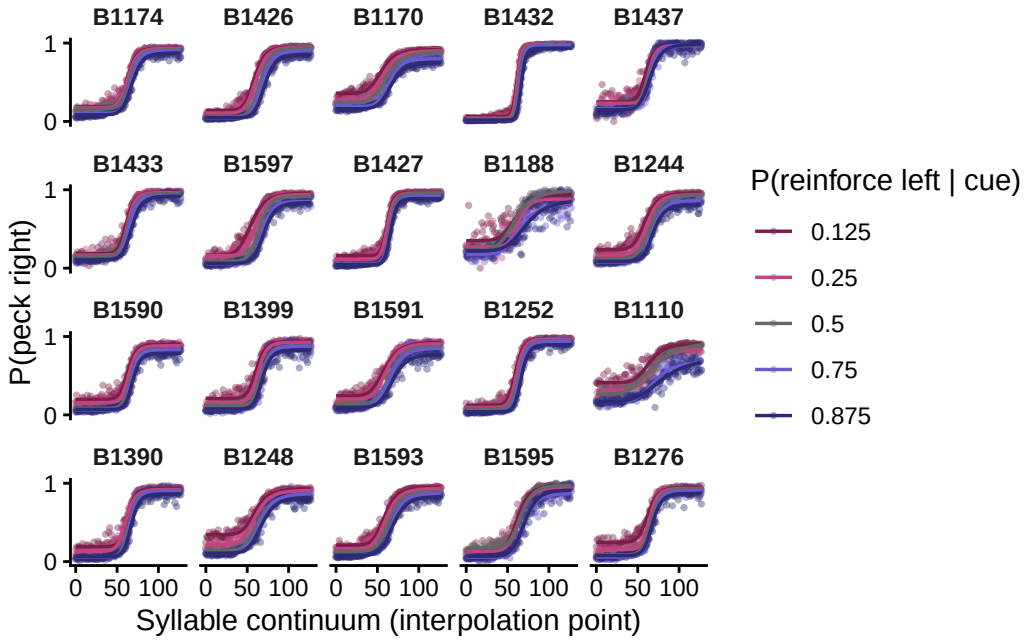
```r
#### Predicted curves
x_grid <- seq(min(plot_data$interpolation_point),
              max(plot_data$interpolation_point),
              length.out = 200)

curve_data <- fits |>
  filter(!map_lgl(model, is.null)) |>
  mutate(pred = map(model, \(m) tibble(
    interpolation_point = x_grid,
    p_right = predict(m, newdata = tibble(interpolation_point = x_grid))
  ))) |>
  select(bird, p_signal_left, pred) |>
  unnest(pred)

#### colors
cue_colors <- c("0.125" = "#7b1e4a", "0.25" = "#c0447f", "0.5" = "gray40",
                "0.75" = "#6a5acd", "0.875" = "#2b2a73")

#### plot
ggplot(plot_data, aes(x = interpolation_point, y = p_right,
                          color = factor(p_signal_left))) +
  geom_point(size = 0.6, alpha = 0.4) +
  geom_line(data = curve_data, linewidth = 0.6) +
  facet_wrap(~ bird, ncol = 5) +
  scale_color_manual(values = cue_colors) +
  scale_y_continuous(limits = c(0, 1), breaks = c(0, 1)) +
  labs(x = "Syllable continuum (interpolation point)",
       y = "P(peck right)",
       color = "P(reinforce left | cue)") +
  theme_classic() +
```

```
  theme(strip.background = element_blank(),
        strip.text = element_text(face = "bold"))
```




```
# fig 3e cleaning

df_all <- read_csv(here("data", "clean", "all_birds_clean.csv"), show_col_types = FALSE)

#### Keep the two strongest cues, informative trials only (L or R direction)
cued <- df_all |>
  filter(cue_prob %in% c(0.75, 0.875), cue_direction %in% c("L", "R"))

#### Proportion of right pecks per bird, morph, cue strength, cue direction, and point
# Assumes the `interpolation` column identifies the morph
cue_summary <- cued |>
  group_by(bird, interpolation, cue_prob, cue_direction, interpolation_point) |>
  summarise(
    p_right  = mean(response_bool),
    n_trials = n(),
    .groups  = "drop"
  )

#### Four-parameter logistic
```

```r
fit_logistic <- function(d) {
  nls(
    p_right ~ p_max + (p_min - p_max) / (1 + (interpolation_point / inflection)^slope),
    data = d,
    weights = n_trials,
    start = list(p_min = 0.1, p_max = 0.9, inflection = 64, slope = 10),
    algorithm = "port",
    lower = c(p_min = 0,   p_max = 0.5, inflection = 1,   slope = 0.1),
    upper = c(p_min = 0.5, p_max = 1,   inflection = 128, slope = 100)
  )
}

#### One fit per bird x morph x cue strength x cue direction
fits <- cue_summary |>
  group_by(bird, interpolation, cue_prob, cue_direction) |>
  nest() |>
  mutate(
    model = map(data, \(d) suppressWarnings(possibly(fit_logistic, otherwise = NULL)(d))),
    inflection = map_dbl(model, \(m) if (is.null(m)) NA_real_ else coef(m)[["inflection"]])
  ) |>
  ungroup()

#### Fit quality: how many failed or landed on a bound?
fits |>
  summarise(
    n_fits     = n(),
    n_failed   = sum(is.na(inflection)),
    n_at_bound = sum(inflection <= 1.5 | inflection >= 127.5, na.rm = TRUE)
  ) |>
  print()
```

```
# A tibble: 1 x 3
  n_fits n_failed n_at_bound
   <int>    <int>      <int>
1    700        2          0
```

```r
#### ΔInflection = left-cue inflection minus right-cue inflection
delta <- fits |>
  filter(!is.na(inflection), inflection > 1.5, inflection < 127.5) |>   # drop failed / at-bo
  select(bird, interpolation, cue_prob, cue_direction, inflection) |>
  pivot_wider(names_from = cue_direction, values_from = inflection) |>
```

```
  drop_na(L, R) |>
  mutate(delta_inflection = L - R)

#### Checks
delta |> count(cue_prob)
```

```
# A tibble: 2 x 2
  cue_prob     n
     <dbl> <int>
1    0.75    175
2    0.875   173
```

```
delta |>
  group_by(cue_prob) |>
  summarise(mean_delta = mean(delta_inflection),
            sd_delta   = sd(delta_inflection))   # means should be POSITIVE
```

```
# A tibble: 2 x 3
  cue_prob mean_delta sd_delta
     <dbl>      <dbl>    <dbl>
1    0.75        5.29     4.54
2    0.875       9.37     9.04
```

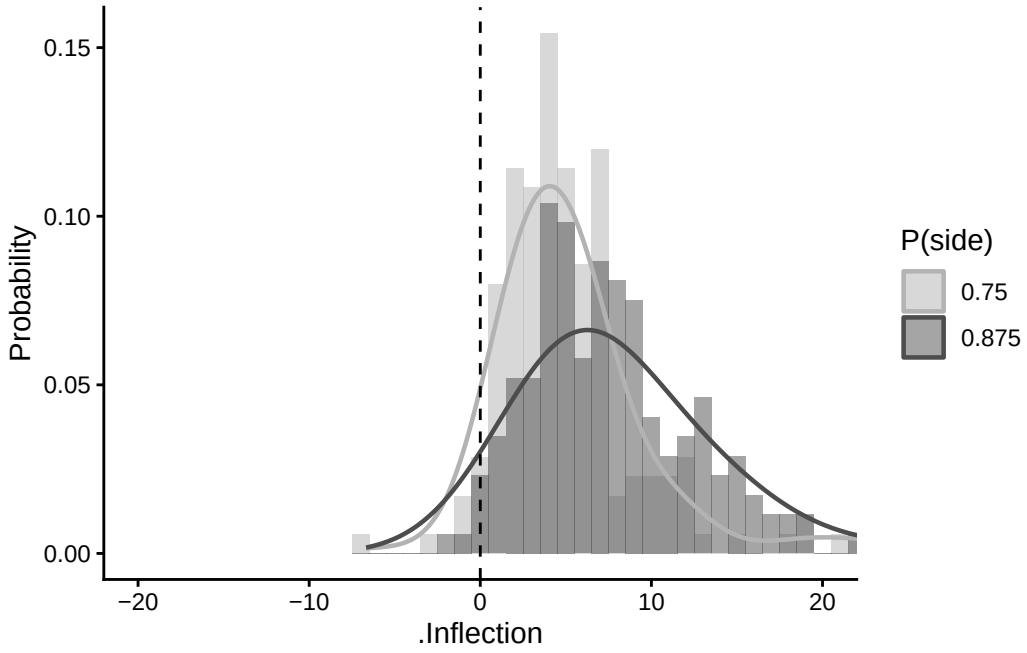
```
write_csv(delta, here("data", "clean", "delta_inflection.csv"))
```

```
#fig 3e plot

binwidth <- 1

ggplot(delta, aes(x = delta_inflection,
                      fill = factor(cue_prob),
                      color = factor(cue_prob))) +
  # y is the proportion of fits in each bin, within each cue strength
  geom_histogram(aes(y = after_stat(density) * binwidth),
                 binwidth = binwidth, position = "identity",
                 alpha = 0.5, color = NA) +
  geom_density(aes(y = after_stat(density) * binwidth),
               fill = NA, linewidth = 0.8, adjust=2) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  scale_fill_manual(values  = c("0.75" = "gray70", "0.875" = "gray30")) +
```

```
  scale_color_manual(values = c("0.75" = "gray70", "0.875" = "gray30")) +
  coord_cartesian(xlim = c(-20, 20)) +
  labs(x = "ΔInflection",
       y = "Probability",
       fill = "P(side)", color = "P(side)") +
  theme_classic()
```




```
# fig 3f cleaning

#### Proportion of right pecks per bird, morph, cue condition, and point
# Assumes the `interpolation` column identifies the morph
cond_summary <- df_all |>
  group_by(bird, interpolation, p_signal_left, interpolation_point) |>
  summarise(
    p_right  = mean(response_bool),
    n_trials = n(),
    .groups  = "drop"
  )

#### Four-parameter logistic
fit_logistic <- function(d) {
  nls(
```

```
    p_right ~ p_max + (p_min - p_max) / (1 + (interpolation_point / inflection)^slope),
    data = d,
    weights = n_trials,
    start = list(p_min = 0.1, p_max = 0.9, inflection = 64, slope = 10),
    algorithm = "port",
    lower = c(p_min = 0,   p_max = 0.5, inflection = 1,   slope = 0.1),
    upper = c(p_min = 0.5, p_max = 1,   inflection = 128, slope = 100)
  )
}

#### One fit per bird x morph x cue condition
fits <- cond_summary |>
  group_by(bird, interpolation, p_signal_left) |>
  nest() |>
  mutate(
    model = map(data, \(d) suppressWarnings(possibly(fit_logistic, otherwise = NULL)(d))),
    inflection = map_dbl(model, \(m) if (is.null(m)) NA_real_ else coef(m)[["inflection"]])
  ) |>
  ungroup()

#### Fit quality: how many failed or landed on a bound?
fits |>
  summarise(
    n_fits     = n(),
    n_failed   = sum(is.na(inflection)),
    n_at_bound = sum(inflection <= 1.5 | inflection >= 127.5, na.rm = TRUE)
  ) |>
  print()
```

```
# A tibble: 1 x 3
  n_fits n_failed n_at_bound
   <int>    <int>      <int>
1    875        2          0
```

```
#### Keep good fits only
fits_clean <- fits |>
  filter(!is.na(inflection), inflection > 1.5, inflection < 127.5) |>
  select(bird, interpolation, p_signal_left, inflection)

#### Checks
nrow(fits_clean)                # the paper reports n = 1,050
```

```
[1] 873
```

```
fits_clean |> count(p_signal_left)
```

```
# A tibble: 5 x 2
  p_signal_left     n
          <dbl> <int>
1         0.125   173
2         0.25    175
3         0.5     175
4         0.75    175
5         0.875   175
```

```
fits_clean |>
  group_by(p_signal_left) |>
  summarise(mean_inflection = mean(inflection))   # should INCREASE with p_signal_left
```

```
# A tibble: 5 x 2
  p_signal_left mean_inflection
          <dbl>           <dbl>
1         0.125            59.8
2         0.25             61.7
3         0.5              64.5
4         0.75             67.0
5         0.875            69.2
```

```
write_csv(fits_clean, here("data", "clean", "inflection_by_cue.csv"))
```

```
# 3f plot

cue_colors <- c("0.125" = "#7b1e4a", "0.25" = "#c0447f", "0.5" = "gray40",
                "0.75" = "#6a5acd", "0.875" = "#2b2a73")

#### Bird-level means (the colored triangles): average over each bird's morphs
bird_means <- fits_clean |>
  group_by(bird, p_signal_left) |>
  summarise(inflection = mean(inflection), .groups = "drop")

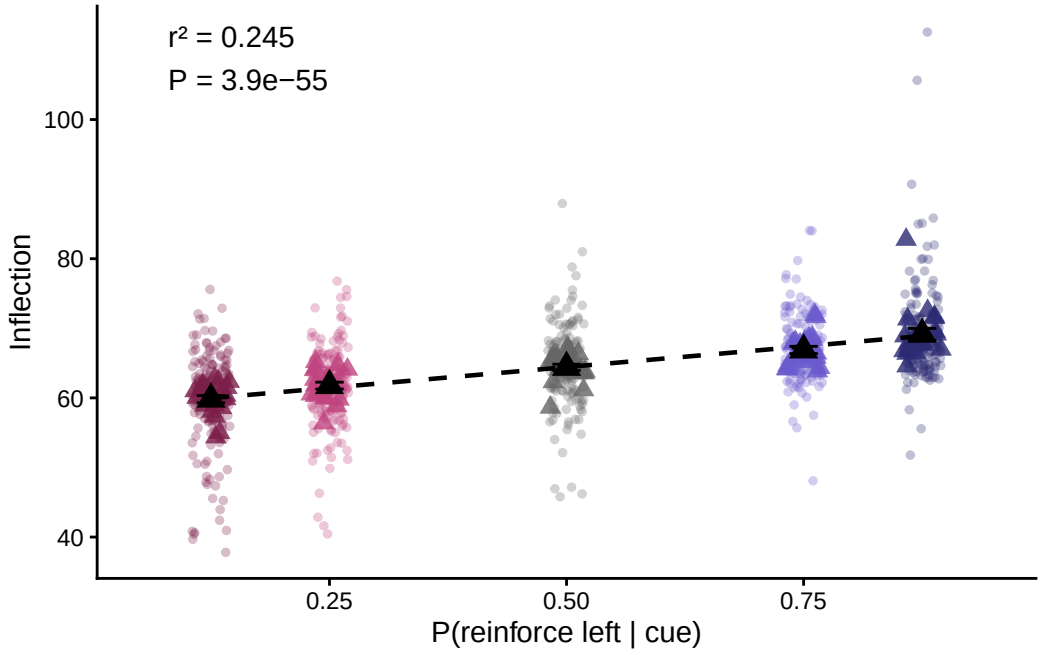
#### Across-bird means with s.e.m. (the black triangles and error bars)
cue_means <- bird_means |>
```

```
  group_by(p_signal_left) |>
  summarise(
    mean_inflection = mean(inflection),
    sem = sd(inflection) / sqrt(n()),
    .groups = "drop"
  )

#### Correlation statistics for the annotation (Pearson, over all morph-level fits)
ct <- cor.test(fits_clean$p_signal_left, fits_clean$inflection)
stat_label <- sprintf("r² = %.3f\nP = %.1e", ct$estimate^2, ct$p.value)

#### Plot
ggplot() +
  # each morph for each bird
  geom_jitter(data = fits_clean,
              aes(x = p_signal_left, y = inflection, color = factor(p_signal_left)),
              width = 0.02, height = 0, size = 1, alpha = 0.3) +
  # each bird, averaged over morphs
  geom_point(data = bird_means,
             aes(x = p_signal_left, y = inflection, color = factor(p_signal_left)),
             shape = 17, size = 2.5, alpha = 0.8,
             position = position_jitter(width = 0.02, height = 0)) +
  # empirical regression line
  geom_smooth(data = fits_clean,
              aes(x = p_signal_left, y = inflection),
              method = "lm", se = FALSE,
              color = "black", linetype = "dashed", linewidth = 0.8) +
  # across-bird means with s.e.m.
  geom_errorbar(data = cue_means,
                aes(x = p_signal_left,
                    ymin = mean_inflection - sem, ymax = mean_inflection + sem),
                width = 0.03, color = "black") +
  geom_point(data = cue_means,
             aes(x = p_signal_left, y = mean_inflection),
             shape = 17, size = 3.5, color = "black") +
  annotate("text", x = 0.08, y = Inf, label = stat_label,
           hjust = 0, vjust = 1.3, size = 4) +
  scale_color_manual(values = cue_colors, guide = "none") +
  coord_cartesian(xlim = c(0.05, 0.95)) +
  labs(x = "P(reinforce left | cue)", y = "Inflection") +
  theme_classic()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Side-by-side graph with original graph is ideal here

Exploratory analyses

Any follow-up analyses desired (not required).

Discussion

Summary of Reproduction Attempt

Open the discussion section with a paragraph summarizing the primary result from the key analysis and assess whether you successfully reproduced it, partially reproduced it, or failed to reproduce it.

Commentary

Add open-ended commentary (if any) reflecting (a) insights from follow-up exploratory analysis of the dataset, (b) assessment of the meaning of the successful or unsuccessful reproducibility attempt - e.g., for a failure to reproduce the original findings, are the differences between original

and present analyses ones that definitely, plausibly, or are unlikely to have been moderators of the result, and (c) discussion of any objections or challenges raised by the current and original authors about the reproducibility attempt (if you contacted them). None of these need to be long.